UNIVERSITY OF CAPE TOWN

Department of Statistical Sciences

Honours Analytics

Assignment 1

Unnati Shankar
Kelvin Wei

Date: 23 March 2026

Table of contents

1 Introduction

This report analyses Cape Town Airbnb listing data from InsideAirbnb with the goal of predicting whether a listing is high or low price. The dataset contains 10000 listings and 28 features; these features include host characteristics (e.g. response time, acceptance rate), listing characteristics (e.g room type, price), and location information (e.g. longitude, latitude).

Given the significant effects of short-term rental sites, such as Airbnb, on housing prices and long-term rental availability in Cape Town, it is imperative to understand the drivers of highly priced listings. `HighPrice` was selected as the target variable as it most directly captures housing affordability, which is the issue motivating this analysis. This binary target variable is defined as:

$$\text{HighPrice} = \begin{cases} 1 & \text{if price exceeds the } 75^{th} \text{ percentile} \\ 0 & \text{otherwise} \end{cases}$$

The data required minimal preprocessing: listing `id` and `host_id` were removed, and `host_since` was converted to `host_tenure_days` - defined as the number of days since the host joined relative to 31 December 2025. The data was then split into a training set (80%) and test set (20%).

Using the training set, the 75th percentile of price was found to be 2715 ZAR per night. The distribution for price is heavily skewed to the right, shown in Figure 1. The threshold identifies the top quarter of the distributions which would most likely be too pricey for long-term renters. This also results in a manageable class imbalance for the models to handle. The models will still learn from the high price observations, but there will still be some effect on prediction which will be seen once the optimal model is fit on the test set.

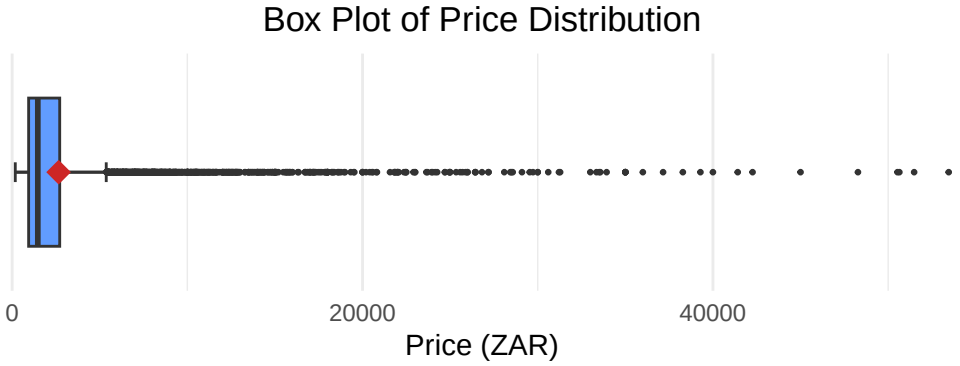

Figure 1: Distribution of Airbnb Listing Prices. The red diamond represents the mean price.

The same threshold of 2715 ZAR was used to define `HighPrice` in the test_set, and the `price` variable was then removed from both sets.

2 Model Development and Selection

To predict and understand the underlying behavior of `HighPrice`, multiple models of varying complexity were fit on the training set. For each model, a broad hyperparameter search space was first defined and

plotted to determine if the initial search space was relatively optimal. If the search space can be seen to be improved then the model was tuned on a new range of hyperparameters.

2.1 Regularised Elastic-net Logistic Regression

Elastic-net regularisation combines L1 (Lasso) and L2 (Ridge) penalties to achieve the best of both worlds: variable selection through the Ridge penalty and dealing with highly correlated variables through the Lasso penalty.

Elastic-net is controlled by two hyperparameters: `alpha`, which determines the mix between L1 and L2 penalties, and `lambda`, which determines the strength of the overall penalty. When $\alpha = 0$, the Elastic-net penalty reduces to Ridge penalty, and when $\alpha = 1$, it reduces to Lasso penalty.

To find the optimal hyperparameter values, we define a search space through which we perform grid search to find the combination of α and λ that yields the highest F1 Score. The search space was initially defined as $\alpha \in [0, 1]$ and $\lambda \in [10^{-4}, 10^{1}]$. However, it was found that larger values of lambda performed worse and we refined our lambda's search space to be $\lambda \in [0, 0.1]$. The plot of the redefined hyperparameter search space is shown in Figure 2.

The F1 Score graph for every mixing percentage (α) is monotonically decreasing with increasing λ; this suggests that optimal values of λ is close to or equal to 0. The optimal value of α is unclear from the graph at $\lambda = 0$ as they all yield in similar F1 Scores. The optimal hyperparameters were by extracted the best model and found to be:

$\alpha = 0.1$; $\lambda = 0$

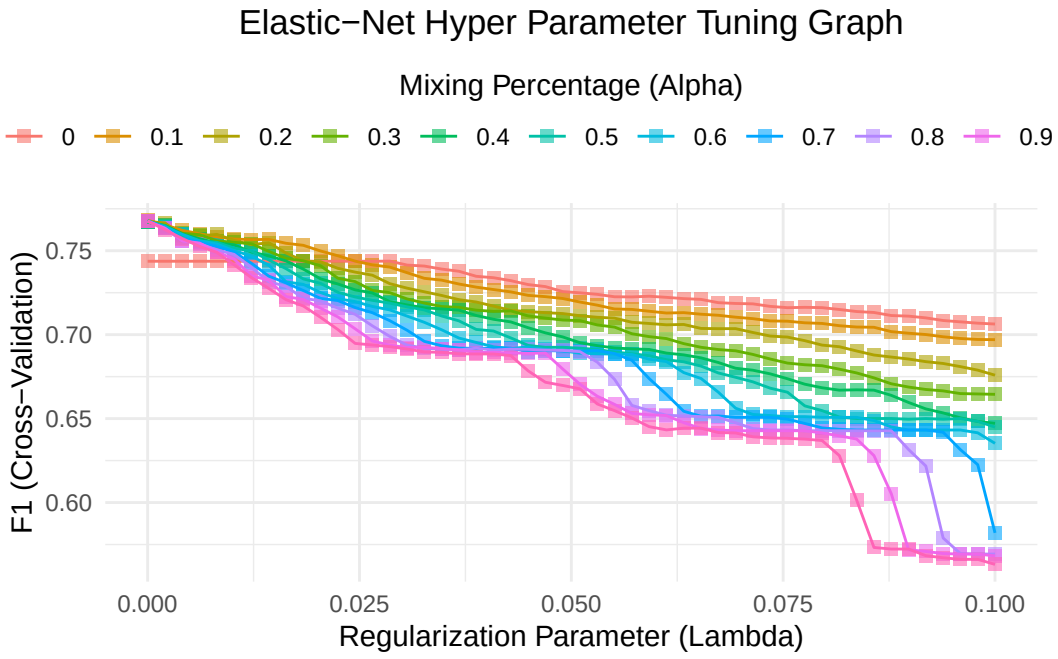

Figure 2: Plot of hyperparameter combinations for elastic net model.

2.2 Random Forest

A Random Forest builds a collection of decision trees, each trained on a bootstrap sample of the data (training set). It accounts for the correlation between trees by considering a random set of features at each split.Random Forests are also controlled by a set of hyperparameters: `mtry` (number of features considered at each split), `splitrule` (the criterion used to evaluate the splits), and `min.node.size` (the minimum number of observations in a terminal node).

A grid search was initially performed over `mtyr` $\in$ {2, 4, 6, 8, 10}, `splitrules` $\in$ {gini, extratrees}, and `min.node.size` $\in$ {2, 4, 6, 8, 10}. Plotting these hyper parameters, we found that the peak hasn't been reached. Hence, we updated our search space to extend to bigger `mtry` parameters (5, 10, 15, 20, 25) while reducing to `min.node.size` parameters (1, 5, 10) to the search space managable. The plot of the refined hyperparameter search space is shown in Figure 3.

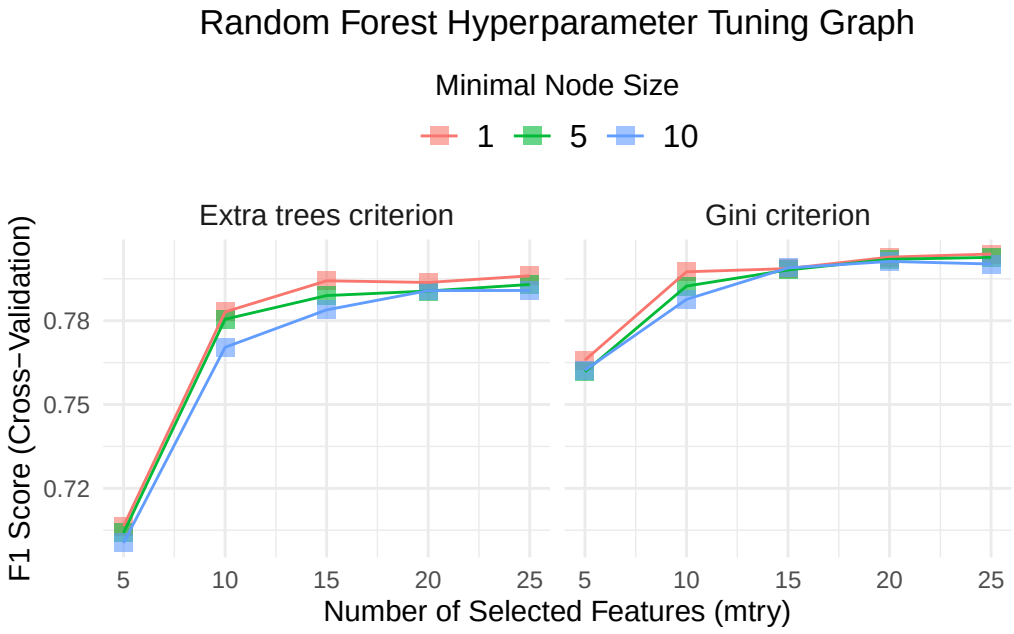

Figure 3: Plot of hyperparameters combinations for random forest model

As seen above, the F1 Score graph for each `min.node.size` is monotonically increasing as `mtry` increases; this suggests that `mtry` will be as high as possible. It can also be seen that the highest F1 Score under the gini `splitrule` is higher than under the extratrees `splitrule`, therefore gini is expected to be optimal. However, it is not very clear from the graphs which `min.node.size` yields the highest F1 Score using gini. The `bestTune` values from the model are:

`mtry` = 25; `splitrule` = gini; `min.node.size` = 1.

2.3 Boosted Tree Model (Gradient Boosting)

Gradient Boosting grows a collection of trees sequentially, as each tree learns from the mistakes from the previous one. This is the opposite of random forests where the trees are grown independently. The hyperparame-

ters controlling Gradient Boosting are: `n.trees` (number of boosting iterations), `interaction.depth` (maximum tree depth), `shrinkage` (learning rate - how much a tree corrects previous tree), and `n.minobsinnode` (minimum number of observations in each terminal node).

The grid search was initially performed over `n.trees` $\in \{100, 200, 300\}$, `interaction.depth` $\in \{2, 4, 6\}$, `shrinkage` $\in \{0.01, 0.1\}$, and `n.minobsinnode` $\in \{5, 10\}$. The results are shown in Figure 4 .

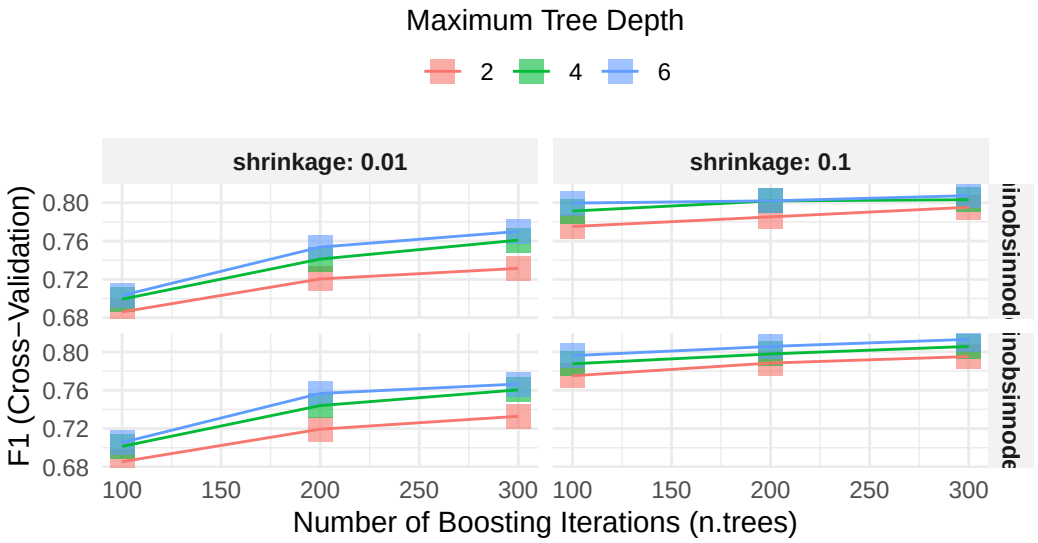

Figure 4: Plot of initial hyperparameters combinations for gradient boosting model

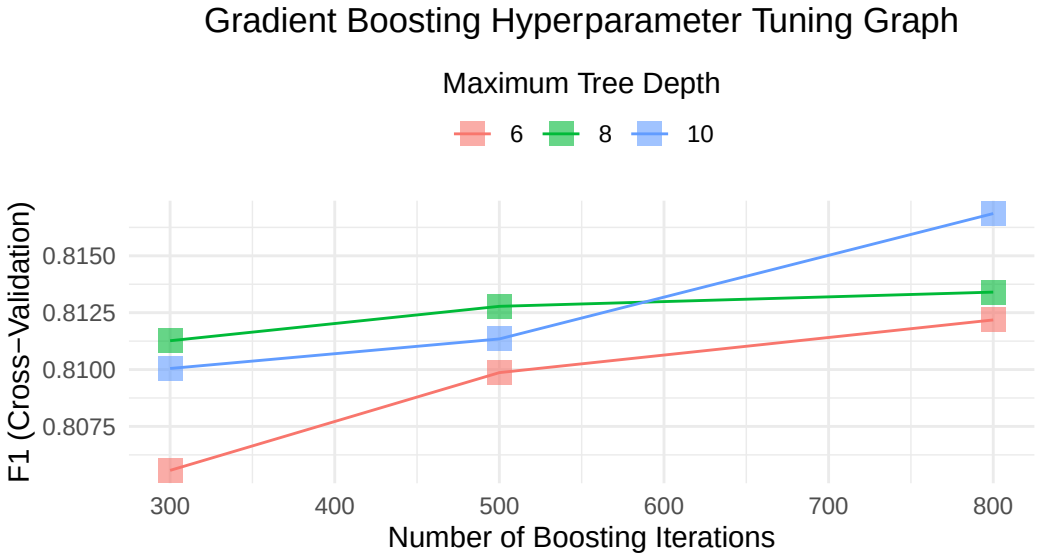

Figure 5: Plot of hyperparameters combinations for gradient boosting model. Constant hyperparameters are learning rate (shrinkage) = 0.1 and min. obs in node = 10.

From preliminary grid search analysis, our initial search space was close to the peak. Hence we decide to expand our search of `n.trees` and `interaction.depth` a little further. We also now fix `shrinkage` to

0.1 as it consistently better than 0.01 and `n.minobsinnode` had little impact on F1 scores so it was fixed at 10. Analysing the plot above, it is evident that `n.trees` = 800 and `interaction.depth` = 10 results in the highest F1 Score. We decide to not expand our hyper parameter search space further to prevent potentially over fitting. Figure 5 shows the hyperparameter search space that was narrowed down and expanded further. The optimal parameters from our model are:

`n.trees` = 800; `interaction.depth` = 10; `shrinkage` = 0.1; `n.minobsinnode` = 10

2.4 K-Nearest Neighbours (KNN)

KNN classifies a new observation by finding the k most similar training observations and taking a majority vote or average. The only hyperparameter is k, the number of neighbours. The features were centered and scaled to ensure that the results are not biased due to scale differences.

Multiple KNN models were run with different combinations of potential variables. The number of variable importance variables from the Random Forest model was used to run KNN models and their F1 Scores were plotted with the number of variables selected. A search from of the range from the first 6 variables selected to 20 variables was performed. This was repeated using permutation based variable importance ranking and F1 scores stored. Figure 6 shows the behaviour of F1 score with different combinations of features.

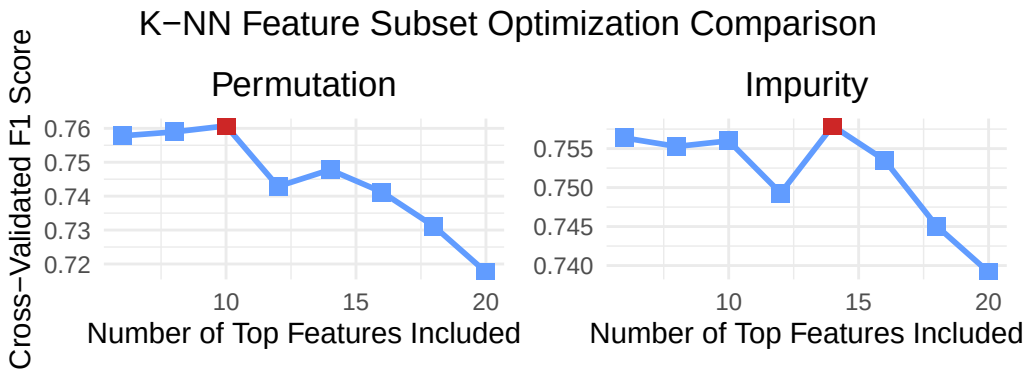

Figure 6: k-NN variable feature selection

It is clear the the first 10 variables from the permutation based importance ranking yielded the highest F1 Score of 0.7607234. A KNN model was then run using the 10 variables as follows: longitude, bathrooms, accommodates, bedrooms, latitude, review_scores_rating, neighbourhood_ward, reviews_per_month, host_listings_count, host_tenure_days. The hyperparameter tuning graph for this knn feature selected model is shown in Figure 7 .

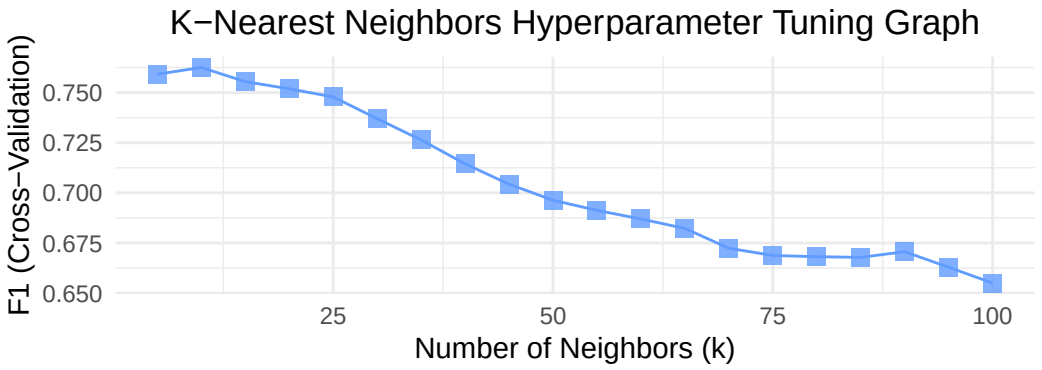

Figure 7: Plot of hyperparameters combinations for feature selected KNN model.

From the above plot, the highest F1 Score yielded is around $k = 10$, but we can confirm exact value by extracting it from the model: 10. We show that variable selection indeed did improve the basic KNN model that uses all variables. A saturated KNN model was trained and it's performance metrics were compared against other models, as discussed in the next section.

2.5 Discussion on Best Model

The table below presents the cross-validated performance metrics for all four models at τ=0.5.

Table 1: Model Performance Comparison with F1 Stability

Model	Accuracy	F1	F1_SD	Precision	Recall	Specificity	AUROC
Elastic-net	0.893	0.769	0.031	0.836	0.711	0.954	0.939
Random Forest	0.908	0.807	0.034	0.850	0.767	0.955	0.955
Gradient Boosting	0.912	0.817	0.021	0.853	0.784	0.955	0.958
K-NN Feature Selected	0.893	0.774	0.026	0.820	0.733	0.947	0.926
K-NN Saturated	0.863	0.695	0.030	0.780	0.626	0.941	0.856

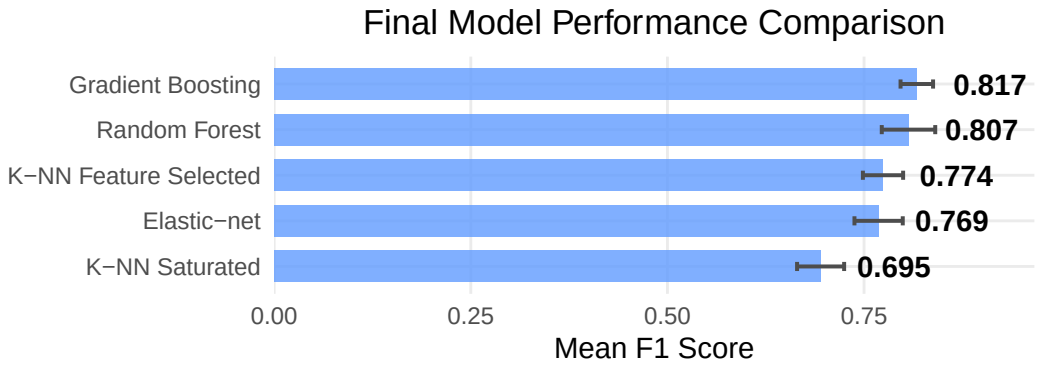

Figure 8: Comparison of Cross-Validated F1 Scores. Error bars represent 1 standard deviation.

Ignoring the F1_SD column for now, all the metrics have noticeably improved from KNN Saturated to KNN Feature Selection, besides specificity which decreased from **0.952** to **0.947**. This is not a very stressful issue since the increase of recall from **0.593** to **0.733** definitely outweighs it.

The Gradient Boosting model performed best across all metrics besides specificity, where it tied with Random Forest.

The Elastic-net model did perform reasonably well with F1 of **0.769** and AUROC of **0.939**, this shows that a linear model can capture meaningful information from the data, just less effectively than tree-based models.

KNN Saturated performed the worst across all the metrics. The low recall of **0.626** suggests that the model struggles to identify true high price listings, most likely due to the curse of dimensionality (KNN uses distances to predict but this method becomes less effective as dimensions increase).

Meanwhile, the KNN Feature Selected performed similar to Elastic-net with the same F1 Score of **0.893**.

Random Forest and Gradient Boosting perform very similarly across all the metrics. Looking at the F1_SD column now, Gradient Boosting has lower F1 standard deviation (**0.021**) than Random Forest (**0.034**). This combination of highest F1 Score and lowest F1 standard deviation is enough to conclude that the Gradient Boosting model is the most accurate and reliable one out of the four models.

3 Threshold Optimization

3.1 Optimal Decision Rule Threshold & CV ROC Curve

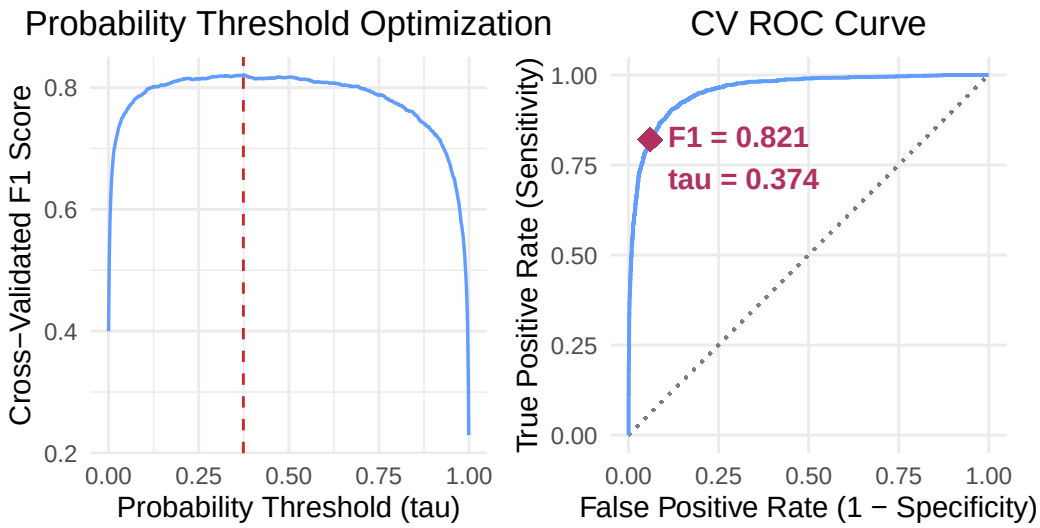

Figure 9: Left: plot of f1 score vs probability threshold (tau). Right: plot of CV ROC curve.

A plot of the performance (F1 Scores) of our Gradient Boosting model for different decision rule thresholds(τ) in the range of 0 to 1 is shown in Figure 9 .The optimal τ is found to be 0.374 with the corresponding F1 Score of 0.8207052.

The specificity and sensitivity metrics corresponding to the optimal τ are 0.9401766 and 0.8209105 respectively. The auroc metric for optimal tau is 1

3.2 Performance of model on Test set

The performance metrics of the Gradient Boosting model on the test set are as follows:

Table 2: Test set metrics

Model	Accuracy	F1	Precision	Recall	Specificity	AUROC
Gradient Boosting	0.897	0.796	0.824	0.771	0.942	0.954

The model accuracy of **0.897** means that the model correctly classified **89.7**% of the listings, which is the large majority of them. The F1 Score of **0.796** suggests a good balance between precision and recall. The precision of **0.824** reflects that **82.4**% of the listings predicted as high price were correctly classified. The recall of **0.771** shows that out of all the high price listings, the model correctly classified **77.1**% of them. Specificity of **0.942** indicates that out of all the low price listings, **94.2**% of them were actually predicted to be low price. Lastly, the AUROC of **0.954** suggests that the model is good at ranking high price listings above low price ones across all thresholds.

The recall is much lower than the precision and specificity; this shows that the model is a bit hesitant in predicting listings as high price. This makes sense as there is a slight imbalance in the `HighPrice` variable.

4 Interpretation and Error Analysis

4.1 Variable importance plots

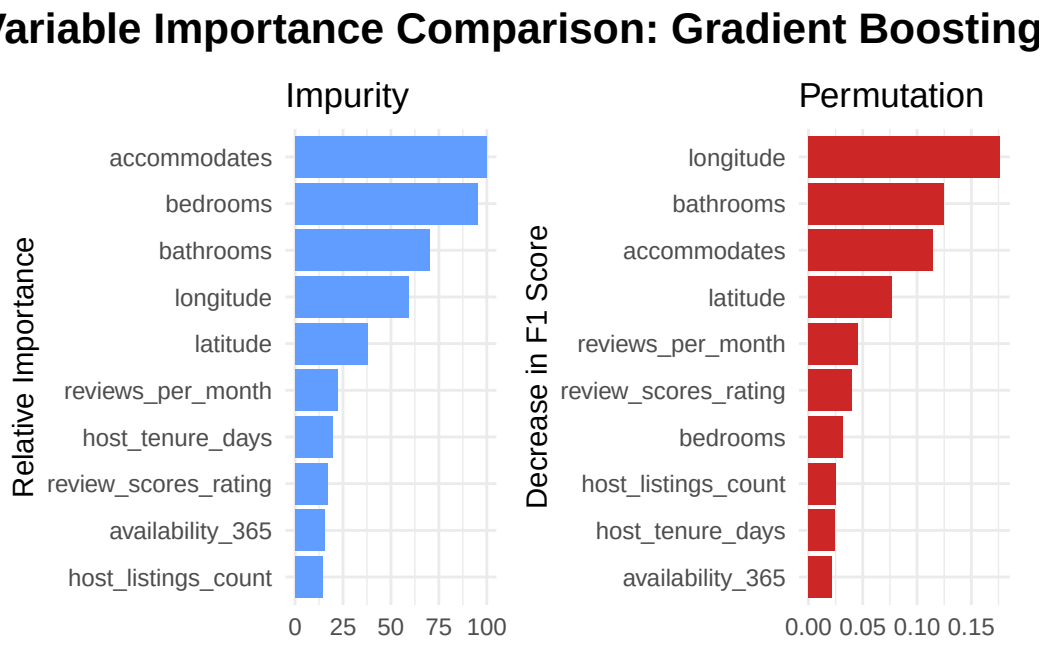

Figure 10: Variable importance comparison graph of gradient boosting. Impurity is based on Gini/Impurity. Permutation is based on F1-Score on a subset of the train set

The above plots present variable importance for the Gradient Boosting model using impurity based importance (left) and permutation based importance (right).

Both the models show that the listing size is extremely influential, with `accommodates` and `bathrooms` appearing in the top four across both plots. This logically makes sense as larger listings can host more guests so they demand higher prices.

The location variables `longitude` and `latitude` also appear to be influential in predicting `HighPrice`. This supports the well-known fact that location is a driver of price. Even though `longitude` ranks first in the permutation plot despite ranking fourth in the impurity plot, suggesting that location may not create as many splits in the trees, but when its information is removed, the model suffers a lot.

4.2 Partial Dependence Plots

Below are the partial dependence plots of `bedrooms`, `bathrooms`, `accommodates` and `longitude`, they present the marginal effect of each variable on the predicted probability of a high price listing, holding all other variables constant:

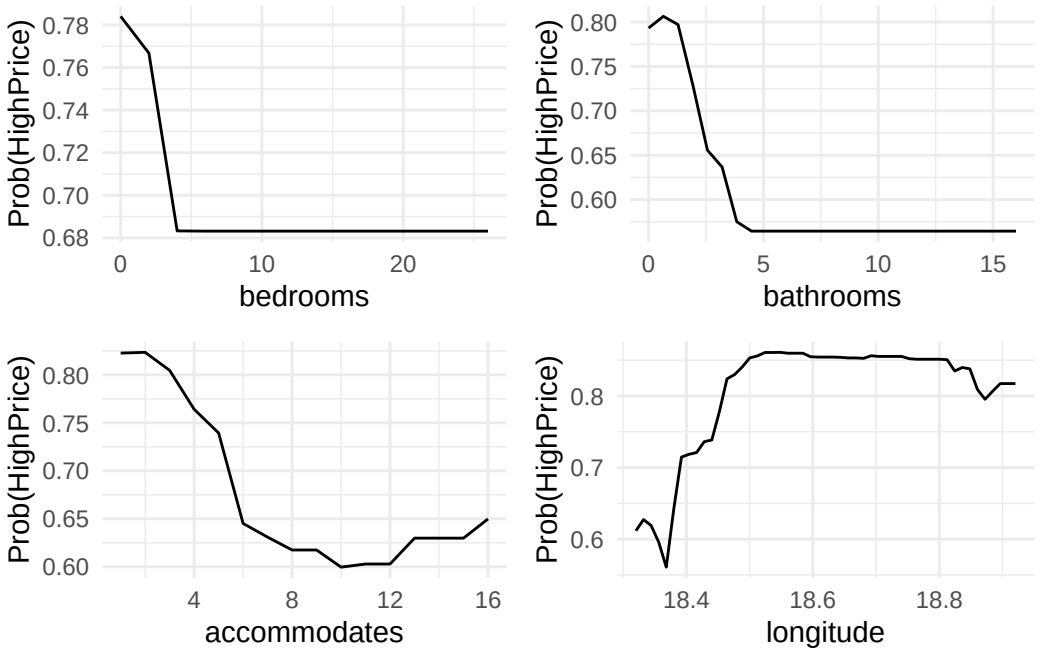

Figure 11: Partial Dependence Plots

`bedrooms`: The probability of being high price decreases suddenly from around 0.78 (listings with 1-2 bedrooms) to around 0.68 (listings with 4 or more bedrooms), and it stays constant. This is an interesting finding since one would expect that more bedrooms would demand higher prices. However, this may reflect that very large listings could be found in lower-cost locations, so the prices are lowered per room to be able to compete with high price listings.

`bathrooms`: The pattern of this plot is similar to that of `bedrooms`', with a probability of around 0.80 (1-2 bathrooms) sharply decreasing to around 0.55 (4 or more bathrooms), and then it stays constant. The reasoning behind this could be similar to `bedrooms`'s reasoning since more `bedrooms` usually means more `bathrooms`.

`accommodates`: The probability decreases from a little over 0.80 (accommodating 1-2 guests) to about 0.60 (accommodating 10-12 guests), then slightly increases until it reaches 0.65 (accommodating 16 guests). This initial drop in probability could be explained by the possibility that larger listings tend to be in low price locations, similar to `bedrooms` and `bathrooms`. It makes sense to pool these three variables together since they represent the size of the listing. However, the increase may be due to the hosts' preferences and them thinking that at some point (around 10), the number of people the listing accommodates outweighs the location, hence price starts increasing.

`longitude`: The probability starts around 0.60 (around longitude 18.3), has a small decline but then rises sharply to around 0.85 (around longitude 18.5), and then remains around 0.85. This suggests that areas to the east are more likely to have high price listings.

4.3 False Positive and Negatives

The table below presents the five most confident false positives and five most confident false negatives from the test set predictions.

Table 3: Error Analysis: 5 FPs and 5 FNs

ErrorType	prob_yes	HighPrice	bedrooms	bathrooms	neighbourhood_ward	room_type
False Positive	0.9985674	No	3	3.0	Ward 54	Entire home/apt
False Positive	0.9977525	No	4	3.0	Ward 59	Entire home/apt
False Positive	0.9922038	No	4	3.5	Ward 61	Entire home/apt
False Positive	0.9891337	No	3	2.0	Ward 23	Entire home/apt
False Positive	0.9872343	No	3	2.0	Ward 73	Entire home/apt
False Negative	0.0005260	Yes	2	1.0	Ward 64	Entire home/apt
False Negative	0.0011158	Yes	1	1.0	Ward 4	Entire home/apt
False Negative	0.0019679	Yes	1	1.0	Ward 61	Entire home/apt
False Negative	0.0028457	Yes	2	2.0	Ward 100	Entire home/apt
False Negative	0.0032084	Yes	2	1.0	Ward 64	Entire home/apt

False Positives: The model predicted high price with extreme probabilities (0.987-0.999) for listings that were actually low price. All the five observations are entire/apartment listings, with 3-4 bedrooms and 2-3.5 bathrooms. These listings have attributes the model associates with high price, but they are not.

False Negatives: On the other hand, the model also predicted low price with extreme probabilities (0.001-0.003) for listings that were actually hihg price. All the observations are again entire/apartment listings, with 1-2 bedrooms and 1-2 bathrooms. This suggests that the model associates smaller listings with lower prices, however, these listings could have been of high price due to their intimate nature geared towards couples.

5 Conclusion

All in all, from the four models used to predict whether a Cape Town Airbnb listing is high price or not (Elastic-net, Random Forest, Gradient Boosting, and K-Nearest Neighbours), the Gradient Boosting model was selected. Given its cross-validated F1 score of **0.817** exceeded the rest, and lowest F1 standard deviation of **0.021**, this model has proved to be the strongest.

Initially, a threshold of $\tau = 0.5$ was used, however, it was found to not be optimal for maximising F1. The optimal threshold was then found to be 0.374, yielding a cross-validated F1 Score of 0.821. Evaluating this model on the test set at this threshold resulted in F1 Score of **0.796** and an AURIC of **0.954**, showing that the model performs well on unseen data.

The variable importance and partial dependence plots reveal that the size of a listing and its location are influential aspects driving high price of the listings.

Through error analysis, it can be concluded that the misclassified predictions were due to listings whose attributes were inconsistent with their price, highlighting a limitation of the model; it can not capture factors that can not be observed, such as host preferences while pricing.